



HASHDIT

HashDit

Smart Contract Security

ListaDao RevenueCollector ssFactory Provenance Security Audit Report

ListaDao

Oct 05 2026 - Oct 06 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	Oct 05 2026 - Oct 06 2026
Overall Risk [Optional]	Informational
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/252
Commit	feat/revenue-collector-ssfactory-provenance

Smart Contracts List

No.	File Name	Verdict	Details
1	RevenueCollector.sol	Informational	[I01]

Findings

[I01] Provenance check does not confirm the collector can claim, diverging from the V2 _validateV2Lp pattern

Contract RevenueCollector.sol

Severity Informational

Description

The V2 redemption path validates both registration and economic ownership: _validateV2Lp requires the pair to be registered in v2Factory and also requires factory.feeTo() == address(this), confirming the collector is the party that receives the fees. The stable swap path is not consistent with this. _validateStableSwapPool only checks that the pool is registered in ssFactory under its own coin pair. It does not confirm that the collector is actually authorized to withdraw fees from the pool.

JavaScript

```
/**
 * @dev Authorizes a pool by provenance instead of a whitelist: it
 * must be registered in
 * `ssFactory` under its own coin pair. Defence in depth only -
 * `withdraw_admin_fees` is
 * MANAGER-gated on the pool, which is the binding gate.
 */
function _validateStableSwapPool(address pool) internal view {
    require(ssFactory != address(0), "ss factory not set");

    IStableSwapFactory.StableSwapPairInfo[] memory infos =
        IStableSwapFactory(ssFactory).getPairInfos(
            IStableSwap(pool).coins(0),
            IStableSwap(pool).coins(1)
        );

    uint256 len = infos.length;
    for (uint256 i = 0; i < len; i++) {
        if (infos[i].swapContract == pool) {
            return;
        }
    }

    revert("invalid stable swap pool");
}
```

Description

Impact

Because `withdraw_admin_fees` is `MANAGER` gated on the pool and pays `msg.sender`, the real condition for a successful claim is that the collector holds the pool's `MANAGER` role, which is the stable swap analog of `feeTo() == address(this)`. Omitting this check produces two consequences.

- First, `previewClaimDexFee` returns fee data for a registered pool even when `claimDexFee` would revert because the collector lacks `MANAGER`, so `preview` and `claim` can disagree, unlike the V2 flow where both share `_validateV2Lp`.
- Second, the failure surfaces as an opaque access control revert from the pool rather than a clear message from the collector. The current behavior is safe because the pool reverts when the collector is not its `MANAGER`, but it is inconsistent with the V2 pattern.

For parity with `_validateV2Lp`, extend `_validateStableSwapPool` to require that the collector holds the pool's `MANAGER` role, for example `require(IAccessControlEnumerable(pool).hasRole(MANAGER, address(this)), "not pool manager")`.

Recommendation

If not, simply ensure that when the upgrade is performed, the `RevenueCollector` is immediately granted the required `MANAGER` role (although role revocation would resurface this issue)

Status

[Open / Fixed / Acknowledged]

Notes

[N01] Missing setSsFactory wiring can freeze dex fee claims after the upgrade

Contract ListaV3FactoryOwner

Description

This change upgrades a live RevenueCollector proxy and removes the whitelist gate that previously authorized stable swap pools, replacing it with provenance checks that read the ssFactory address. On a freshly upgraded or newly deployed proxy, ssFactory is address(0) because initialize does not set it.

Description

Impact

As a result, _validateStableSwapPool reverts with "ss factory not set", and every claim path that depends on it stops working: claimDexFee, batchClaimDexFees, and previewClaimDexFee all revert. No deployment or wiring script sets ssFactory, so until the Timelock holding DEFAULT_ADMIN_ROLE calls setSsFactory with the StableSwapFactory address, admin fee collection is fully blocked. This creates a window where protocol revenue cannot be claimed.

[N02] StableSwap fee claim path collects no revenue after PR #251

Contract ListaV3FactoryOwner

Description

[PR #251](#) reduces StableSwapPool to pro-rata liquidity only and removes swap functionality. As a result, no admin or swap fees accrue on these pools anymore. withdraw_admin_fees now only sweeps tokens that are transferred into the pool directly, meaning dust or donations, and admin_balances returns approximately 0.

Description

The RevenueCollector dex fee claim path, made up of _claimDexFee, claimDexFee, batchClaimDexFees, and previewClaimDexFee, along with the PR #252 ssFactory provenance gating in _validateStableSwapPool, therefore no longer collects any real revenue stream from these pools.

If swaps migrate to the V2/V3 DEX, that revenue is already captured by the collector through redeemV2Lp and batchAccrueVaultFees, and the StableSwap pools become pure collateral baskets.